

RFP 질의응답 시스템

복잡한 RFP 문서를 빠르고 정확하게 분석하여 원하는 정보를 제공합니다.

CODEIT AI 엔지니어 9기
중급 프로젝트 1팀

①

프로젝트 개요

②

데이터 분석

③

Baseline 모델
선정

④

성능 개선 및 평가

⑤

비용 효율성 분석 및
확장 전략

⑥

결론

기존 RFP 분석의 문제점



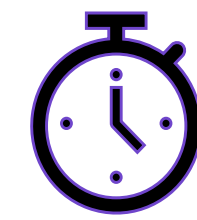
방대한 문서 구조

수백 페이지에 달하는 복잡한 RFP 문서는 핵심 정보를 추출하는 데 과도한 시간과 인력을 소요하게 합니다.



수작업의 한계

문서별 형식 차이로 인해 자동화가 어렵고, 수작업 분석 시 정보 누락 및 분석 부하가 가중됩니다.



응답 지연

제안서 작성에 필수적인 정보 검색 속도가 느려져 프로젝트의 전반적인 생산성이 저하됩니다.

1. 프로젝트 개요

프로젝트 핵심 목표

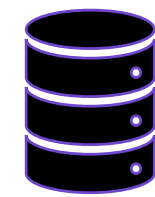
- 💡 **Chunking 전략별 성능 비교:** RFP 구조에 특화된 최적의 문서 분할 단위 및 계층 구조 탐색
- 💡 **LLM & Embedding 모델 벤치마킹:** 데이터 도메인에 가장 적합한 모델 조합 선정
- 💡 **Retrieval/Generation 성능 분석:** 정확도(Accuracy)와 지연 시간(Latency)의 균형점 확보
- 💡 **최적의 파이프라인 구축:** 가이드에 부합하는 RFP 전용 RAG Pipeline 정립 및 검증

기술 스택 및 인프라



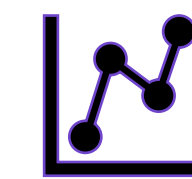
Cloud & Compute

Google Cloud Platform (GCP)
기반의 서버 및 인프라 매니지먼트



Data & Versioning

DVC (Data Version Control)를
활용한 데이터 및 임베딩 버전 관리



MLOps & Tracking

Weights & Biases (W&B)를
통한 실험 결과 추적 및 성능
시각화

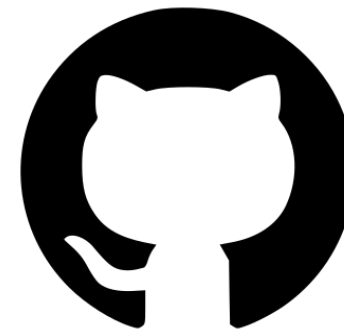
1. 프로젝트 개요

협업 프로세스 및 워크플로우



Communication

Discord 및 Notion을 통한 실시간
지식 공유 및 일정 관리



GitHub Code Collaboration

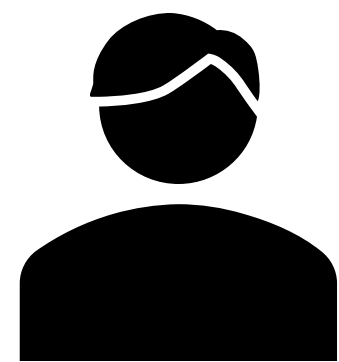
Git 브랜치 전략 및
코드 리뷰 프로세스를 통한 안정적인
코드 협업 구현



Figma Daily Standup

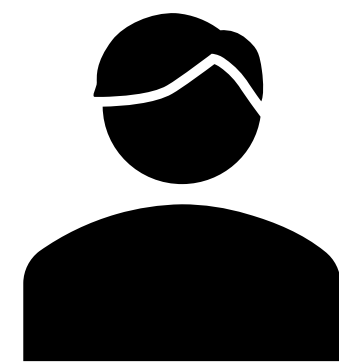
Figma 보드를 활용한
유기적인 데일리 스탠드업 미팅

1팀 조직 및 역할 구성



김경제
팀장 / Evaluation

전체 일정 관리 및
파이프라인 성능 평가



김영성
Data Specialist

RFP 데이터 전처리
및 데이터베이스 관리



신희정
Retrieval Load

검색 엔진 및
임베딩 모델 최적화



엄지영
Retrieval Load

검색 엔진 및
임베딩 모델 최적화



황예원
Generation Load

LLM 프롬프트
엔지니어링 및 답변 생성

2. 데이터 분석

☑ 데이터 분포

총 문서 수

100 개

- HWP : 96 개 / PDF : 4개
- 메타데이터 칼럼: 12개

총 표 개수

9,012개

- 문서당 평균 약 90개
- 구조 보존 최우선 필요

총 이미지 개수

2,502 개

- 문서당 평균 약 25개
- 시각 정보 추출 대상

☑ 문서 특징

⚠ 100개 문서 내 9,012개의 방대한 표 존재

→ 표 구조를 완벽히 보존하는 Markdown Chunking 도입

⚠ HWP 포맷 비율 96%로 매우 높음

→ 데이터 추출 효율화를 위해 전량 PDF 포맷으로 일괄 전환 파이프라인 구축

⚠ 기존 data_list.csv 텍스트 보존율 심각(0.2%~6.0%)

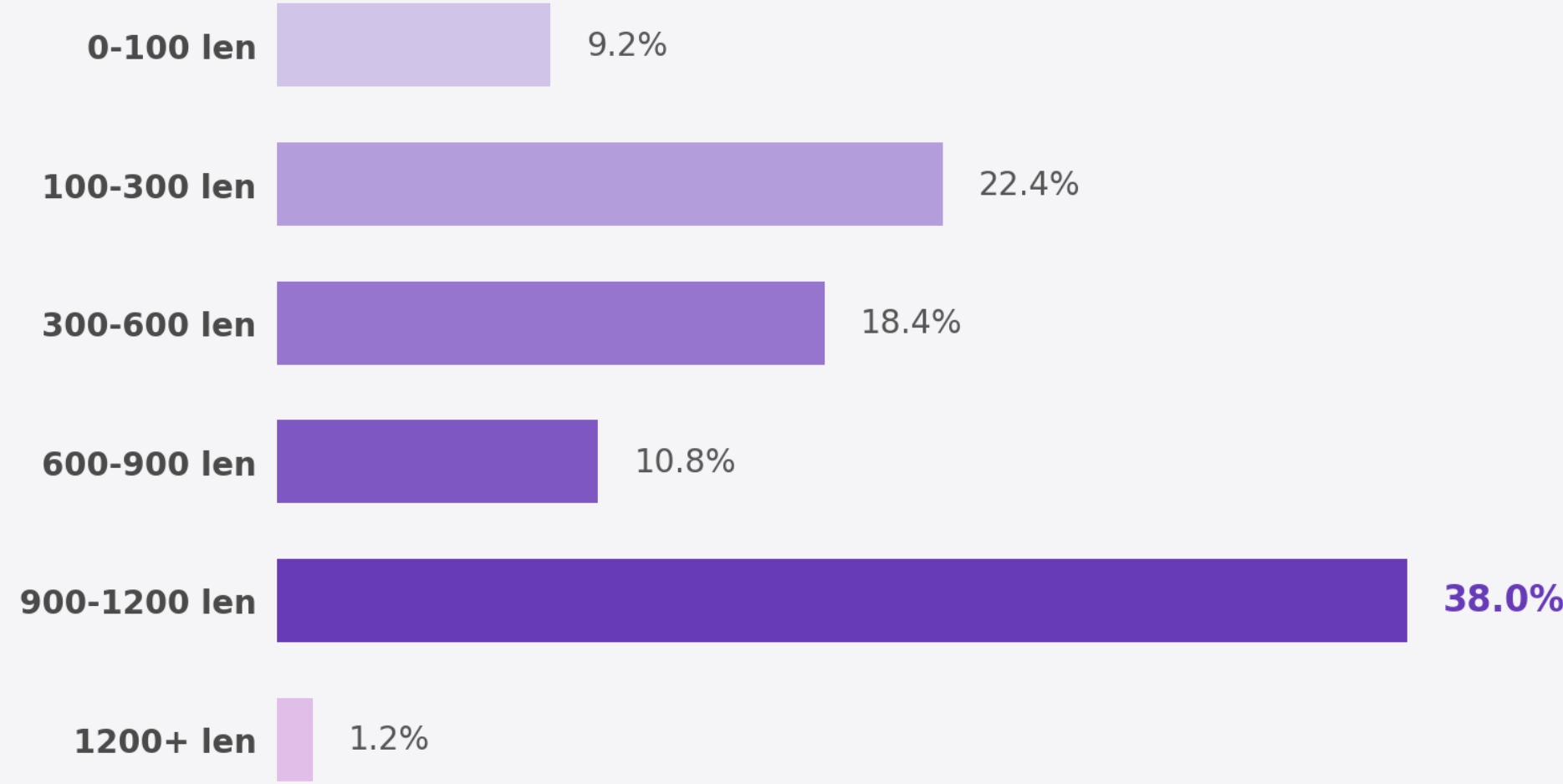
→ 원본 파일에서 직접 텍스트 추출 가공

데이터 전처리

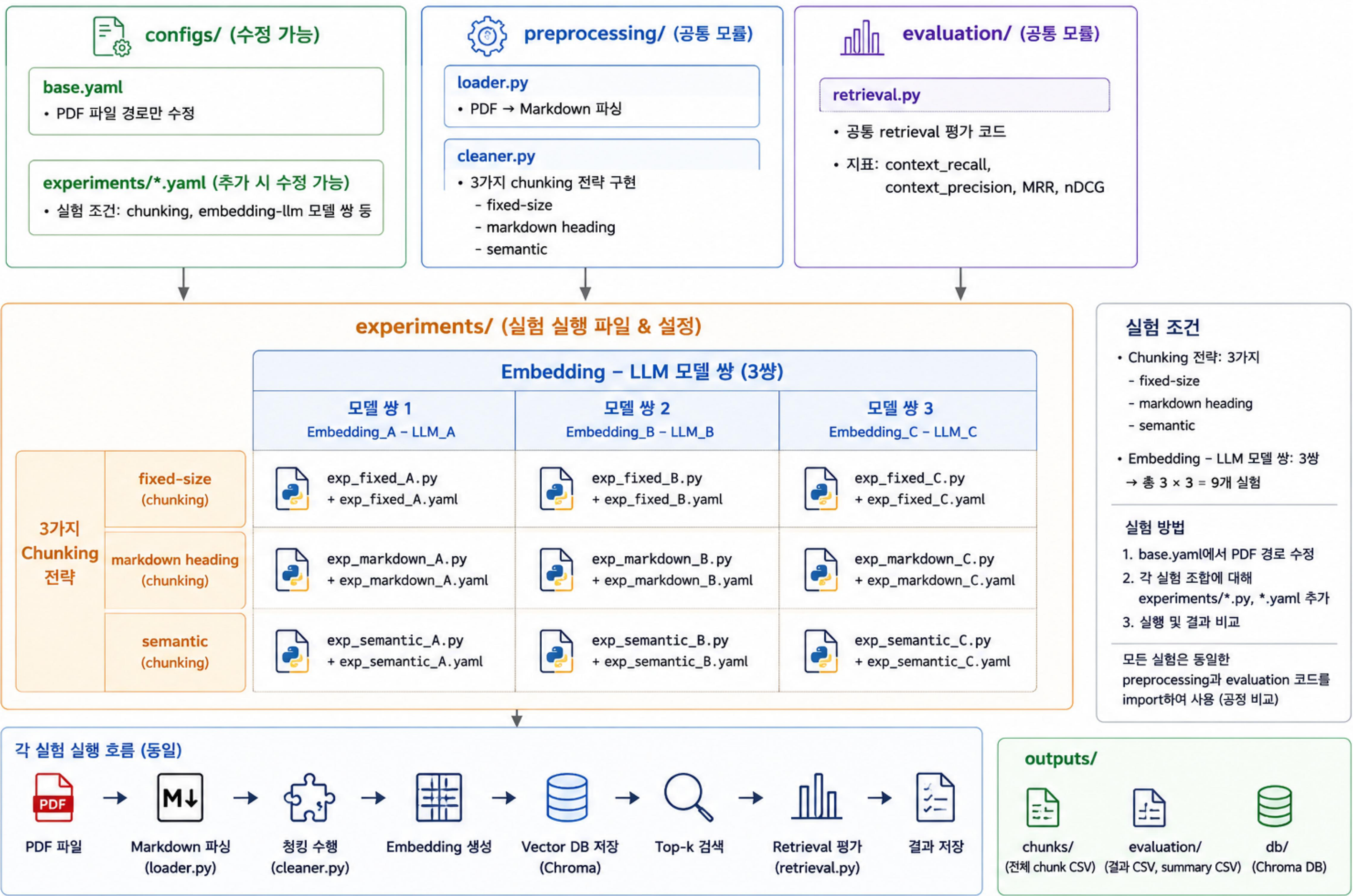
👍 청킹 결과 (markdown heading, chunk size : 900, overlap : 270 기준)

[고려대학교_차세대 포털·학사 정보시스템 구축사업] [제 안 요 청 서 > **. 고려대학교 차세대 포털 학사 정보시스템 구축 사업**] ## **. 고려대학교 차세대 포털 학사 정보시스템 구축 사업** |정보시스템|주요내용|비고| |---|---|---|
- |포털시스템| 통합로그인, 통합/지능형 검색, 마이페이지, 공지/알림, 일정관리,
커뮤니티, 게시판, 사용자별 정보서비스, 위젯, 연계서비스(웹메일,
챗봇, 전자결재, 학사/행정 서비스) 등
학생(졸업생포함), 교직원, 연구원 등 내부 구성원 대상 포털
학생/교수 등 신분별개인별 주요 정보 제공
학사/행정/연구 시스템의 주요 기능 직접 접근
위젯 등 개인화-SSO(기존 보유, 활용), 간편인증, 2차인증적용
사용자 통합관리-포털, 홈페이지, 교내 정보시스템의 웹 페이지
및 메뉴, 첨부 문서 검색
검색자의 신분에 맞는 검색 우선 노출
각 시스템 메뉴(화면) 색인을 통한 검색 및 시스템 접근 연계
사용자 입력 검색어와 검색 결과의 유사도 / 검색 결과 추천
개인화
서비스| |모바일서비스| 통합로그인, 통합/지능형 검색, 마이페이지, 공지/알림, 일정관리,
대학정보, 공간/기기 예약, 맞춤/추천정보, 학사정보, 행정정보 등
모바일 포털

항목	값
총 청크 수	22,018개
평균 길이	609.5자



3. Baseline 모델 선정



3. Baseline 모델 선정

청킹 전략

1. Fixed Size

- 전체 텍스트 정리
- chunk_size 기준으로 앞에서부터 자름
- 각 chunk 앞에 project_name prefix 추가

2. Markdown Heading

- Markdown heading(#, ##, ### 등) 기준으로
섹션 분리
 - 각 섹션 정리
 - 섹션이 chunk_size보다 길면
chunk_size 기준으로 추가 분할
- 각 chunk 앞에 project_name과
section title prefix 추가

3. Semantic

- base_chunks 리스트를 입력으로 받음
- 각 chunk를 정리
- RecursiveCharacterTextSplitter로
separator를 고려해 분할
- chunk_size와 chunk_overlap을
splitter 설정으로 사용

모델 구성

구분	모델쌍 1	모델쌍 2	모델쌍 3
임베딩 모델	text-embedding-3-small	bge-m3	nomic-embed-text-v2-moe
생성 모델	gpt-5-mini	EXAONE-3.5-7.8B-Instruct	Qwen3-8B

3. Baseline 모델 선정

실험 환경

- 대상: PDF 문서 5건
- 벡터 DB: Chroma
- Overlap: 100
- Top-k: 4

평가 목적

- 복잡한 표 및 구조화된 정보의 검색 성능 검증
- 구조화 문서 기반 QA 성능 평가

평가 방법

- 각 문서마다 10개의 QA 평가셋 구축
- QA: 질문(question) + 정확한 정답(ground_truth)

평가 지표

Retrieval

- | | |
|------------------------|--------------------------------|
| • Context Precision | 검색된 상위 리스트 중 유의미한 문서의 상단 밀집도 |
| • Context Recall | 정답 생성에 필요한 필수 소스 정보의 전반적 포함 여부 |
| • MRR | 최초로 '올바른 정답' 한 가지를 찾기까지의 속도 |
| • nDCG | 다등급 관련성 가중치를 적용한 리스트 정렬의 총합 품질 |
| • Embedding Build Time | 임베딩 구축 시간 (초) |

Generation

- | | |
|----------------------|---|
| • Faithfulness | 주어진 문맥에만 기반하여 환각 없이 사실적으로 작성되었는지 평가 |
| • Answer Relevance | 질문의 의도와 목적에 부합하며,
불필요한 정보 없이 정확하게 답변했는지 평가 |
| • ROUGE-L | 정답과의 문장 구조적 유사성 및 정보 결합도를 텍스트 매칭으로 평가 |
| • Generation Seconds | 질문당 답변 생성 시간 (초) |

3. Baseline 모델 선정

Baseline 모델 선정 결과

Embedding model
BAAI/bge-m3

LLM model
Qwen3-8B

Chunking
Markdown chunking

1. Retrieval: bge-m3 성능 우위

- 실험 결과 OpenAI Embedding 대비 bge-m3 조합의 Retrieval 지표가 더 높게 측정됨
- 전략: bge-m3를 기준으로 Retrieval 목표 성능 재정의 및 최적화 진행

2. Generation: OpenAI 벤치마크 설정

- 종합적인 답변 안정성 및 품질은 OpenAI 조합이 가장 우수함
- 전략: Qwen3-8B의 성능 개선 타겟을 OpenAI 수준으로 설정하여 고도화

3. 경제성: Self-hosting 및 확장성 확보

- 대량 처리 시 GPU 서버 임대가 OpenAI API 비용보다 경제적일 것으로 분석됨
- 전략: 성능 근접도에 따른 비용 손익분기점(BEP) 수치 제시 예정

4. 라이선스: 상업적 이용 적합성 확인

- BAAI/bge-m3: MIT License (상업적 이용 가능)
- Qwen3-8B: Apache 2.0 License (상업적 이용 가능)

4. 성능 개선 및 평가

ML Experiment Tracker

All Experiments

Status

Calendar

Platform Analysis

4. 성능 개선 및 평가

1) Max New Token

Max New Token (A100)	Retrieval				
	MRR	nDCG	Context Precision	Context Recall	Embedding Time
256	0.86	0.93	0.84	0.67	19.9
512 (Baseline)	0.85	0.93	0.82	0.67	10.7
1024	0.86	0.93	0.84	0.67	19.9



4. 성능 개선 및 평가

1) Max New Token

Max New Token (A100)	Generation			
	Faithfulness	Answer Relevance	Rouge I	Generation time
256	0.89	0.85	0.32	5.3
512 (Baseline)	0.86	0.81	0.46	9.4
1024	0.84	0.83	0.51	12.5



4. 성능 개선 및 평가

2) LLM SIZE

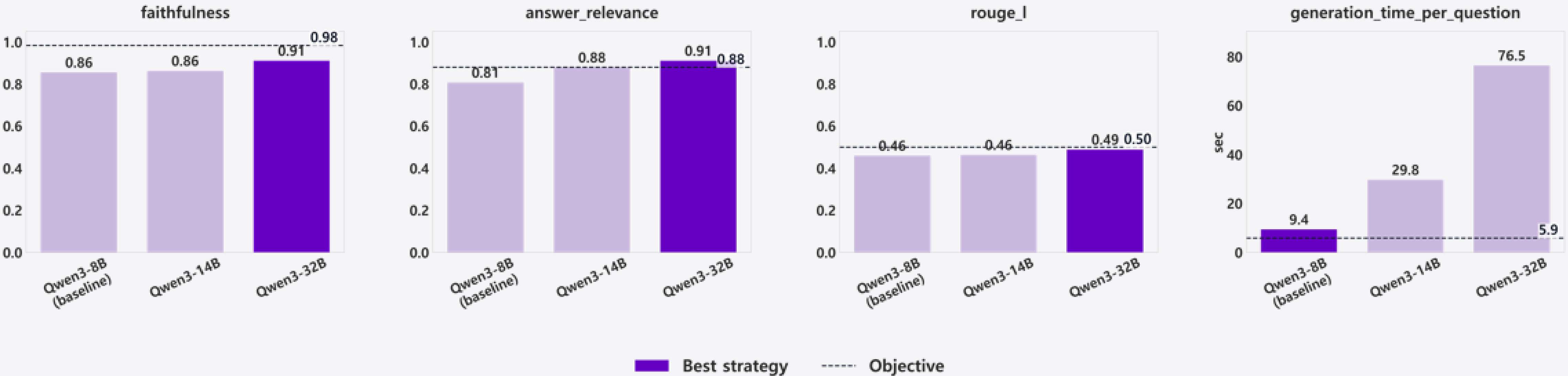
LLM size (A100)	Retrieval				
	MRR	nDCG	Context Precision	Context Recall	Embedding Build Time
Qwen3-8B (baseline)	0.85	0.93	0.82	0.67	10.7
Qwen3-14B	0.85	0.93	0.82	0.67	28.9
Qwen3-32B	0.86	0.93	0.83	0.67	28.7



4. 성능 개선 및 평가

2) LLM SIZE

LLM size (A100)	Generation			
	Faithfulness	Answer Relevance	Rouge I	Generation time per question
Qwen3-8B (baseline)	0.86	0.81	0.46	9.4
Qwen3-14B	0.86	0.88	0.46	29.8
Qwen3-32B	0.91	0.91	0.49	76.5



4. 성능 개선 및 평가

3) 4bit 양자화

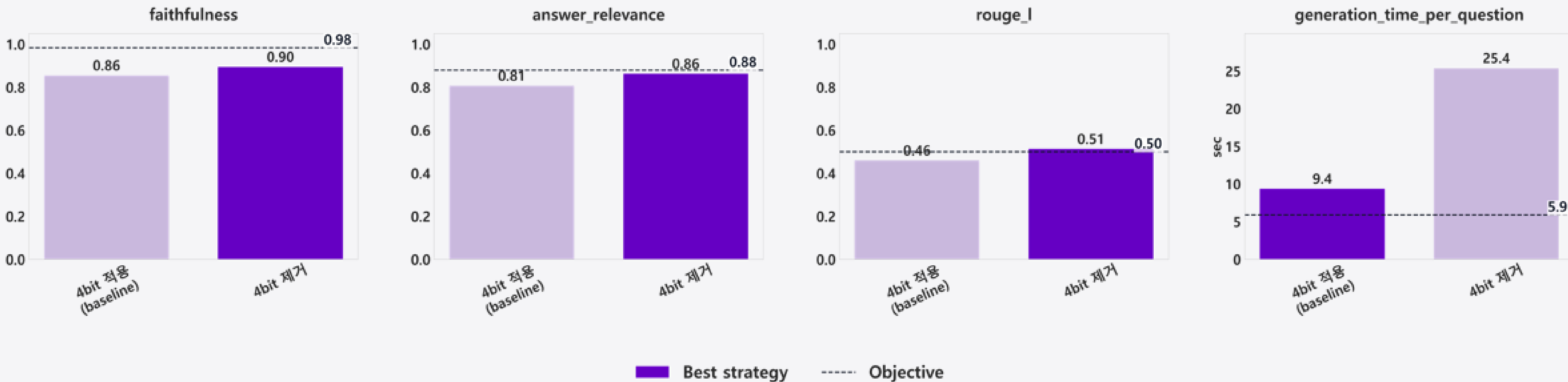
4bit (A100)	Retrieval				
	MRR	nDCG	Context Precision	Context Recall	Embedding Build Time
4bit 적용	0.85	0.93	0.82	0.67	10.7
4bit 제거	0.85	0.93	0.82	0.67	28.9



4. 성능 개선 및 평가

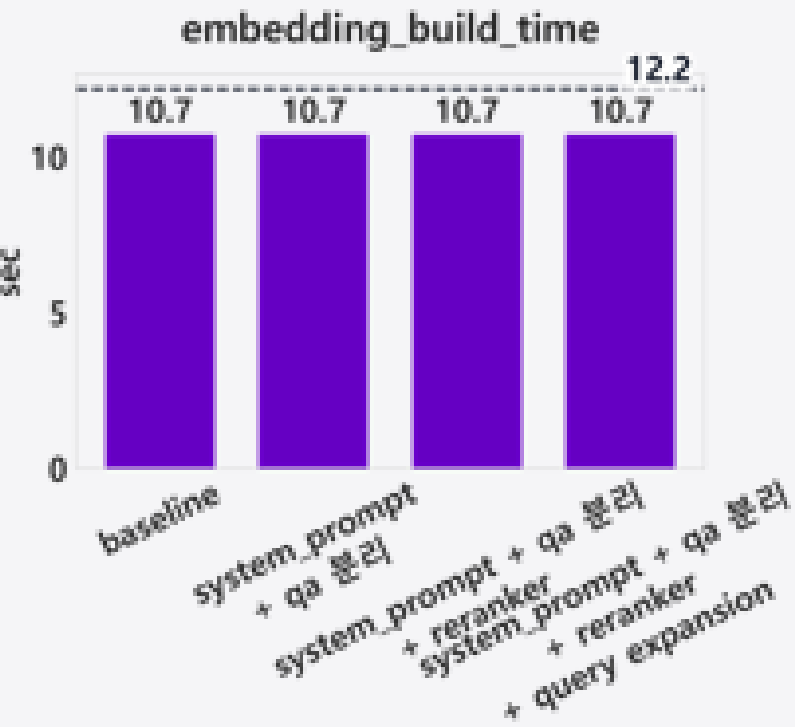
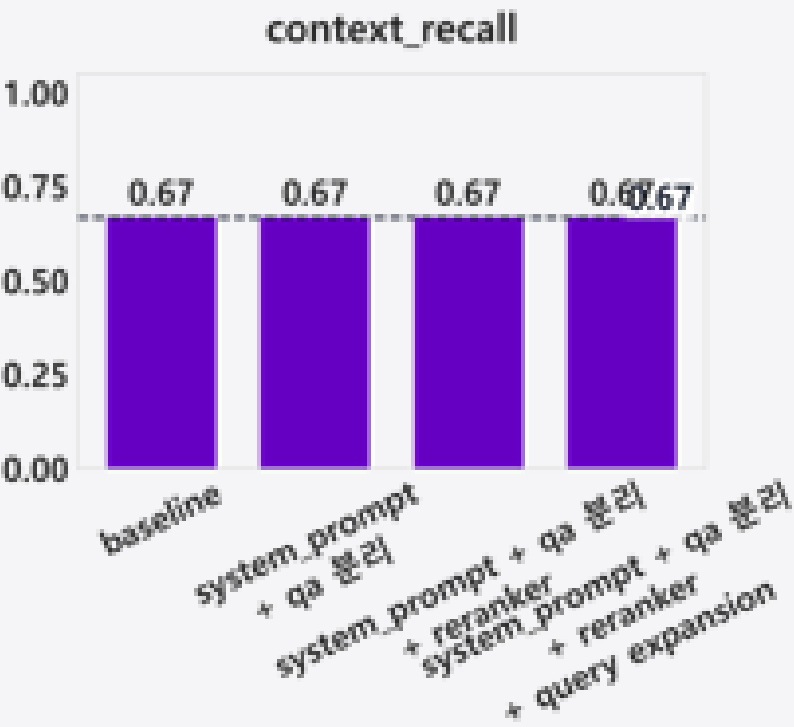
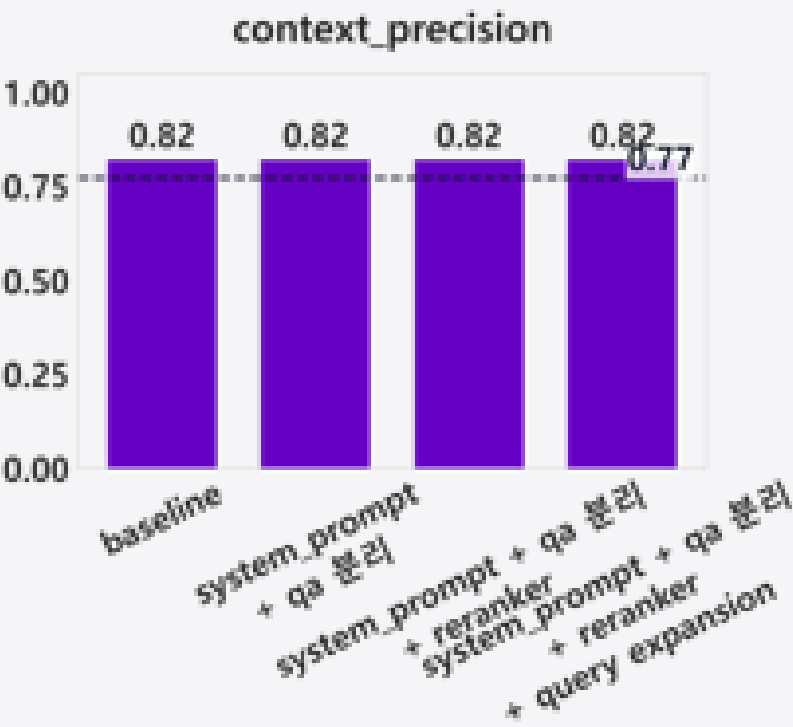
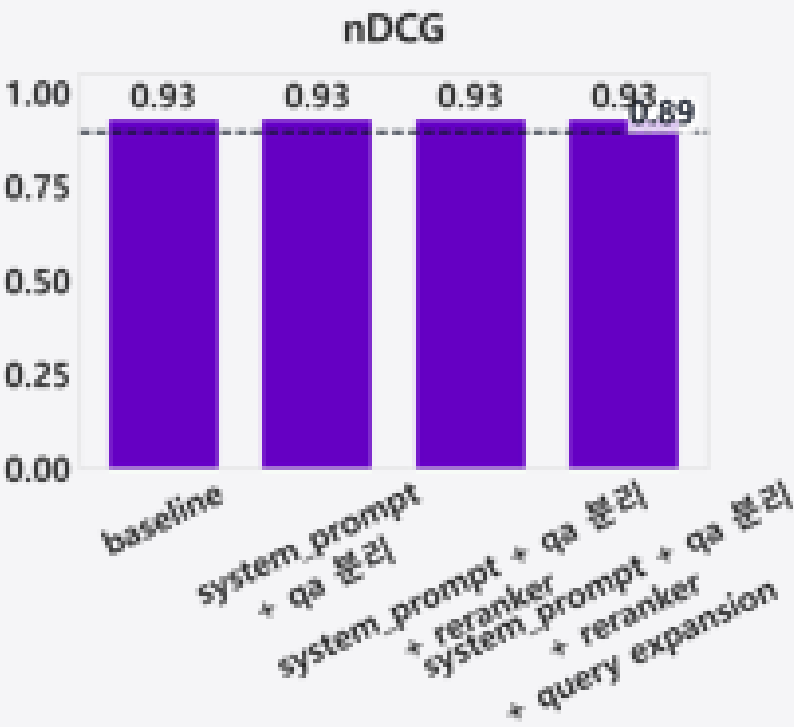
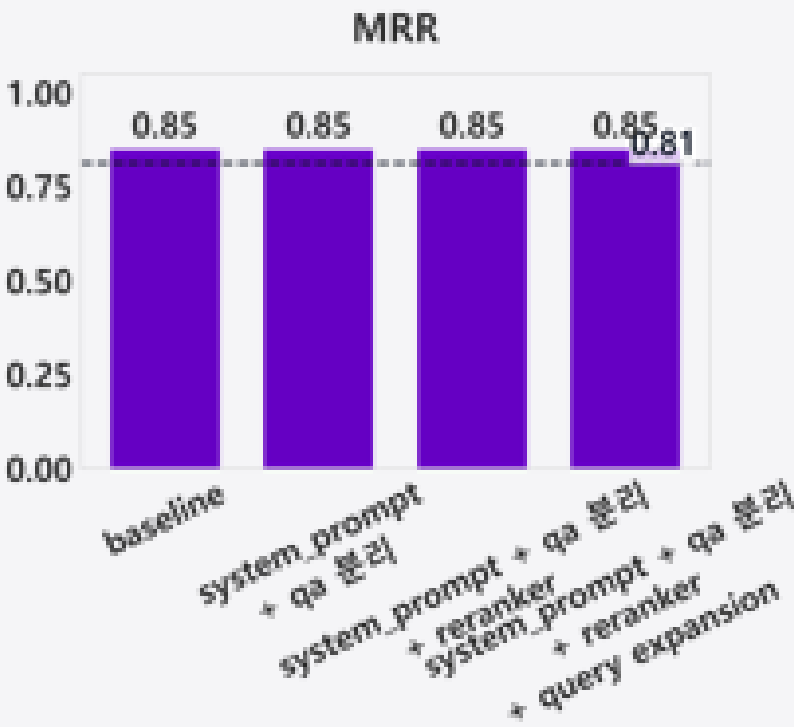
3) 4bit 양자화

4bit (A100)	Generation			
	Faithfulness	Answer Relevance	Rouge I	Generation time per question
4bit 적용	0.86	0.81	0.46	9.4
4bit 제거	0.90	0.86	0.51	25.4



4) Prompt 및 Reranker

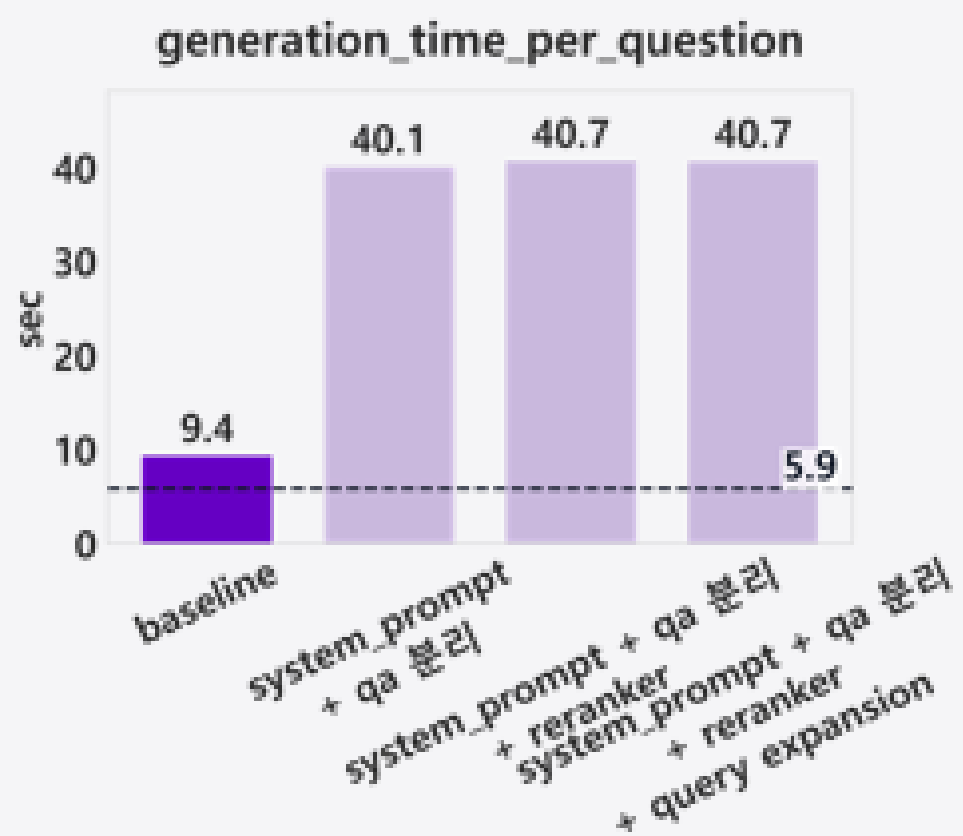
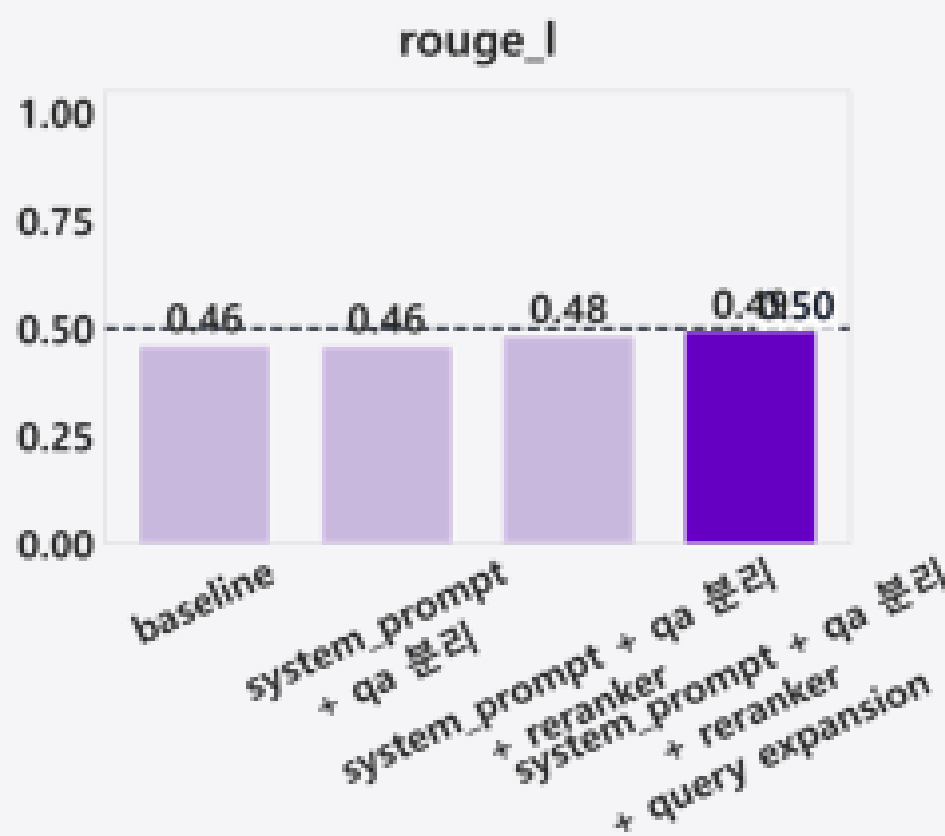
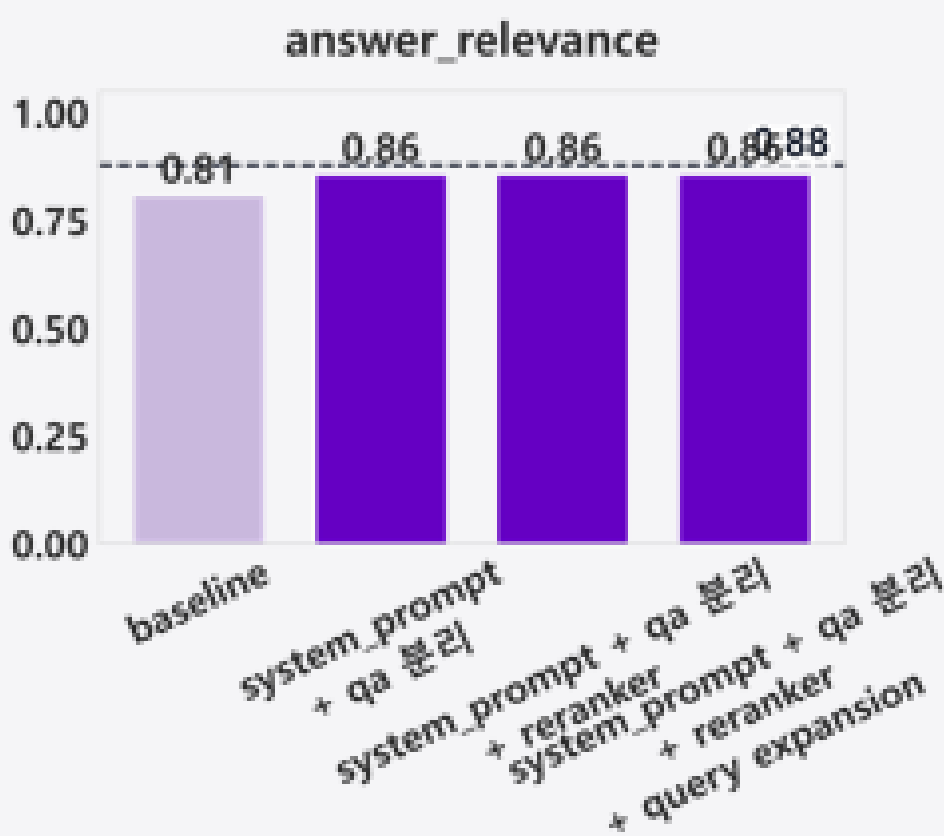
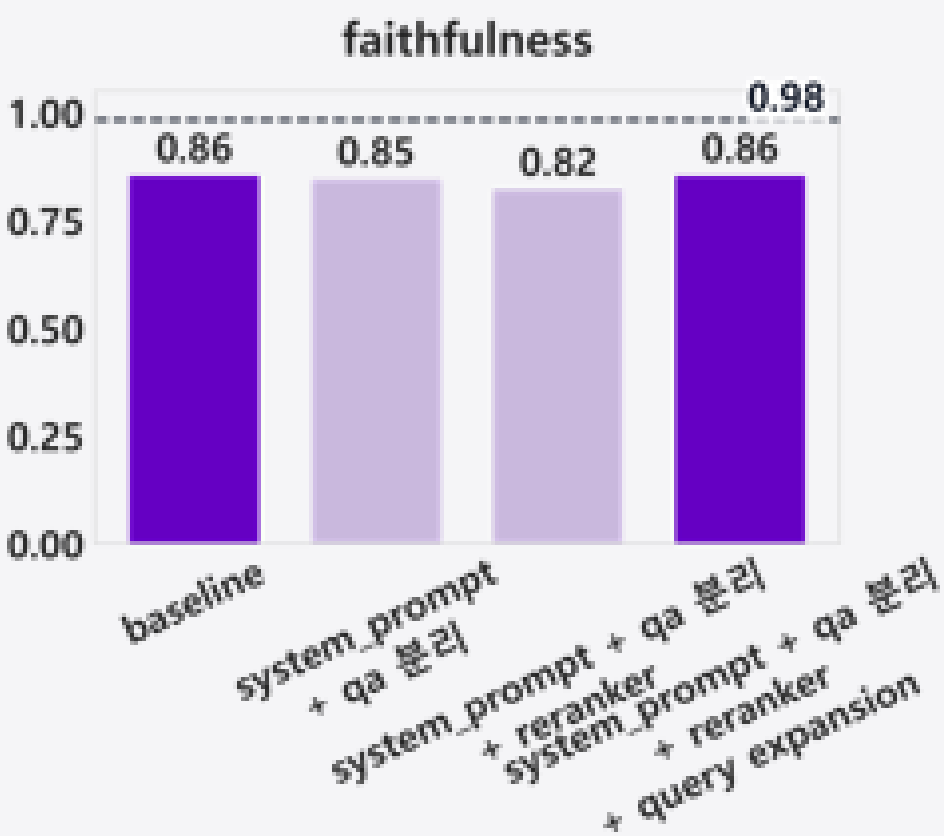
Prompt 및 Reranker Strategy (A100)	Retrieval				
	MRR	nDCG	Context Precision	Context Recall	Embedding Build Time
baseline	0.85	0.93	0.82	0.67	10.7
system_prompt + qa 분리	0.85	0.93	0.82	0.67	10.7
system_prompt + qa 분리 + reranker	0.85	0.93	0.83	0.67	10.7
system_prompt + qa 분리 + reranker + query expansion	0.85	0.93	0.82	0.67	10.7



Best strategy Objective

4) Prompt 및 Reranker

Prompt 및 Reranker Strategy (A100)	Generation			
	Faithfulness	Answer Relevance	Rouge I	Generation time per question
baseline	0.86	0.81	0.46	9.4
system_prompt + qa 분리	0.85	0.86	0.46	40.7
system_prompt + qa 분리 + reranker	0.82	0.86	0.48	40.7
system_prompt + qa 분리 + reranker + query expansion	0.86	0.86	0.49	40.7



4. 성능 개선 및 평가

5) Top - K

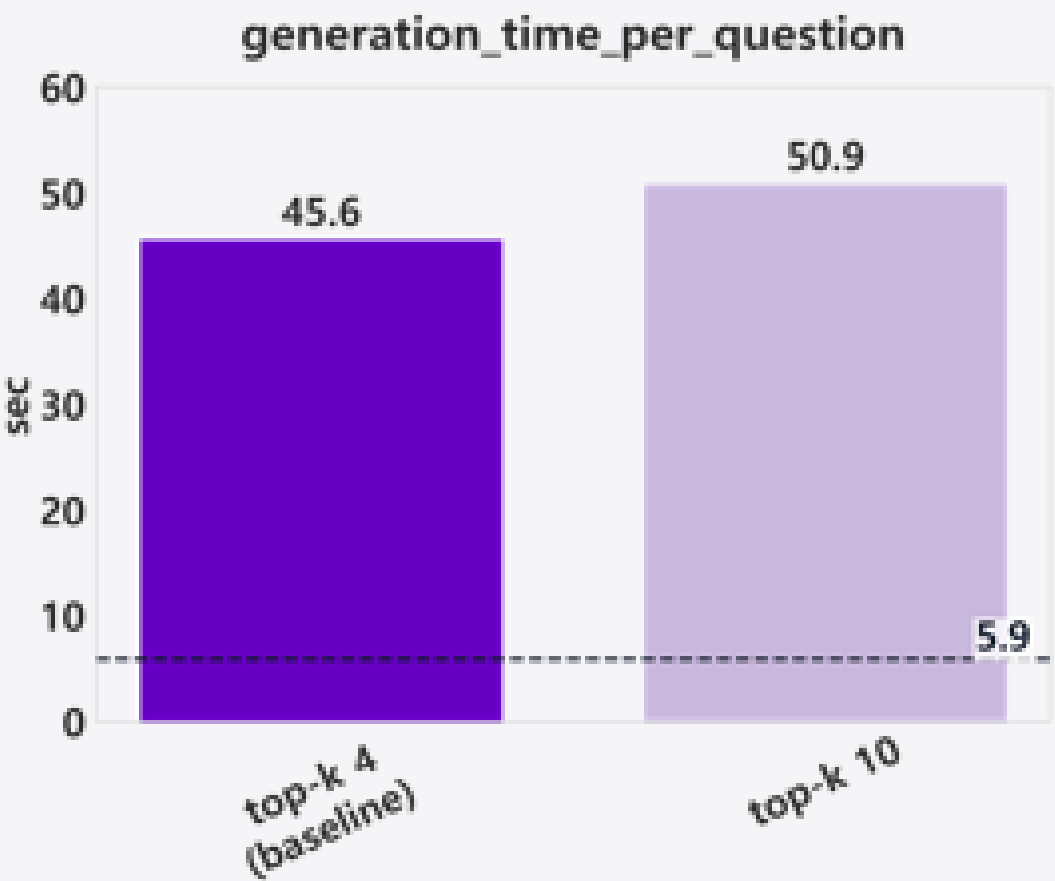
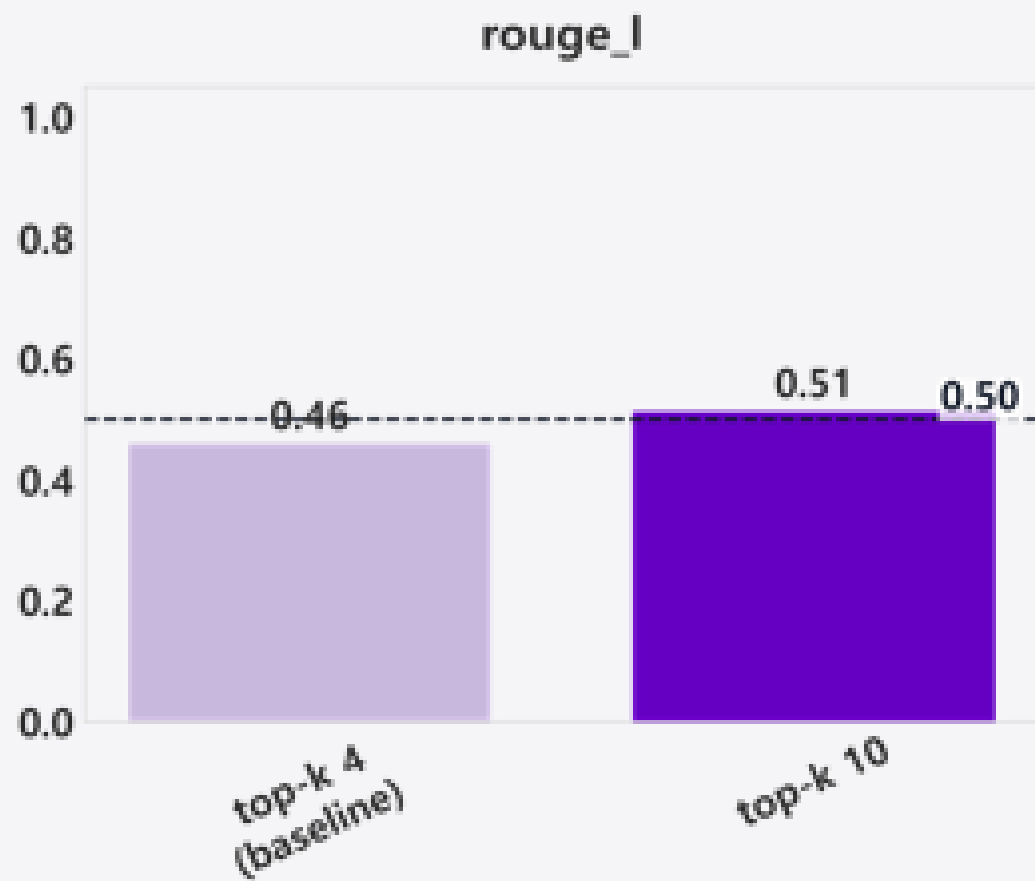
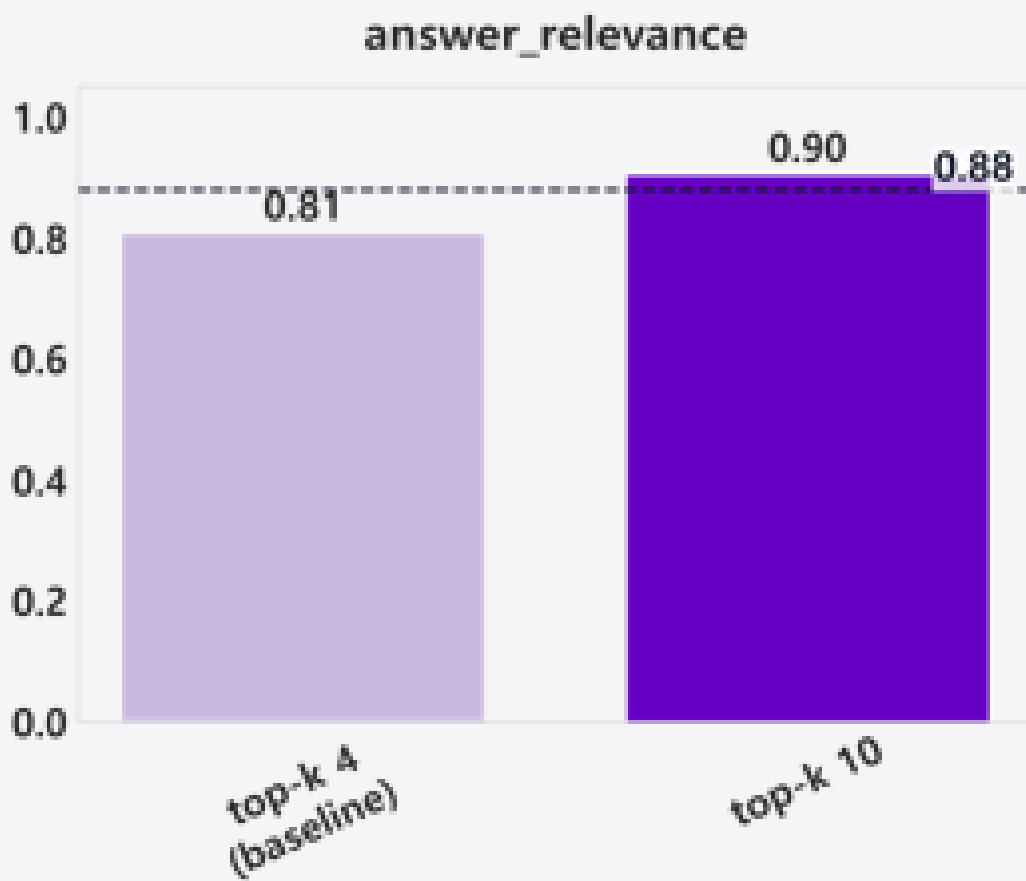
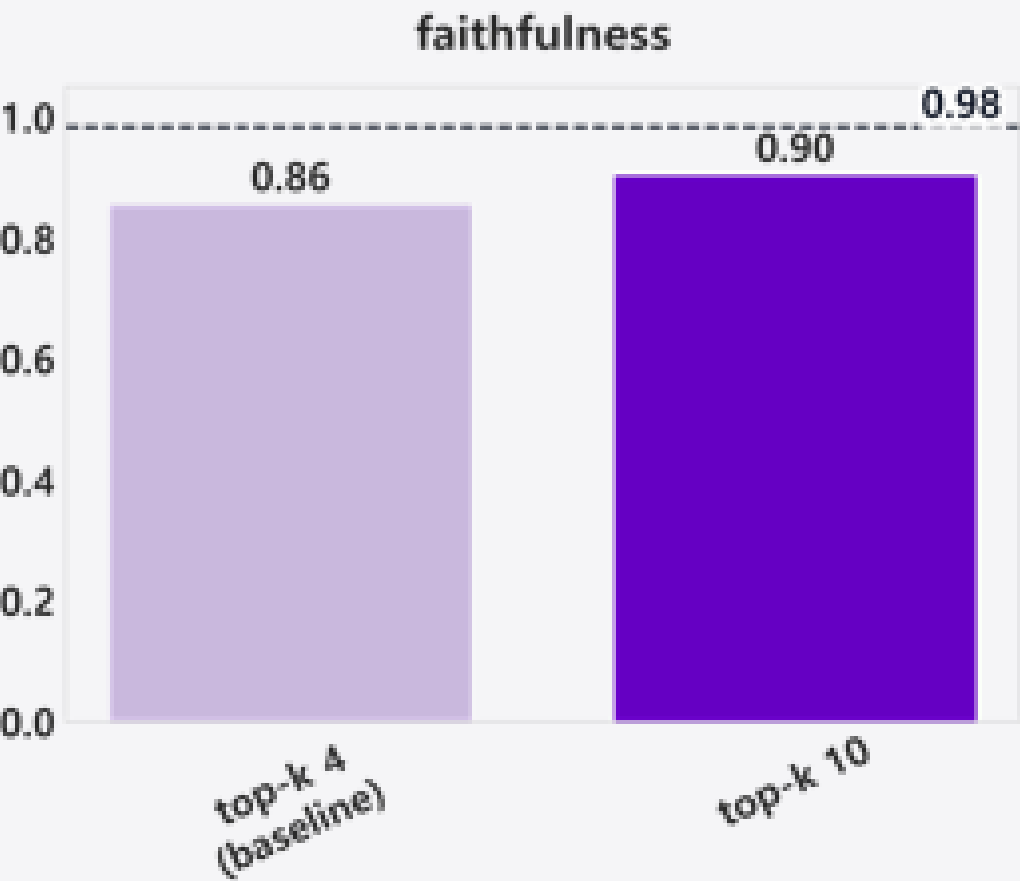
Top-k (L4)	Retrieval				
	MRR	nDCG	Context Precision	Context Recall	Embedding Build Time
Top-4 (baseline)	0.85	0.93	0.82	0.67	55.6
Top-10	0.86	0.91	0.77	0.73	52.0



4. 성능 개선 및 평가

5) Top - K

Top-k (L4)	Generation			
	Faithfulness	Answer Relevance	Rouge I	Generation time per question
Top-4 (baseline)	0.86	0.81	0.46	45.6
Top-10	0.90	0.90	0.51	50.9



Best strategy Objective

4. 성능 개선 및 평가

6) Chunk size / Overlap

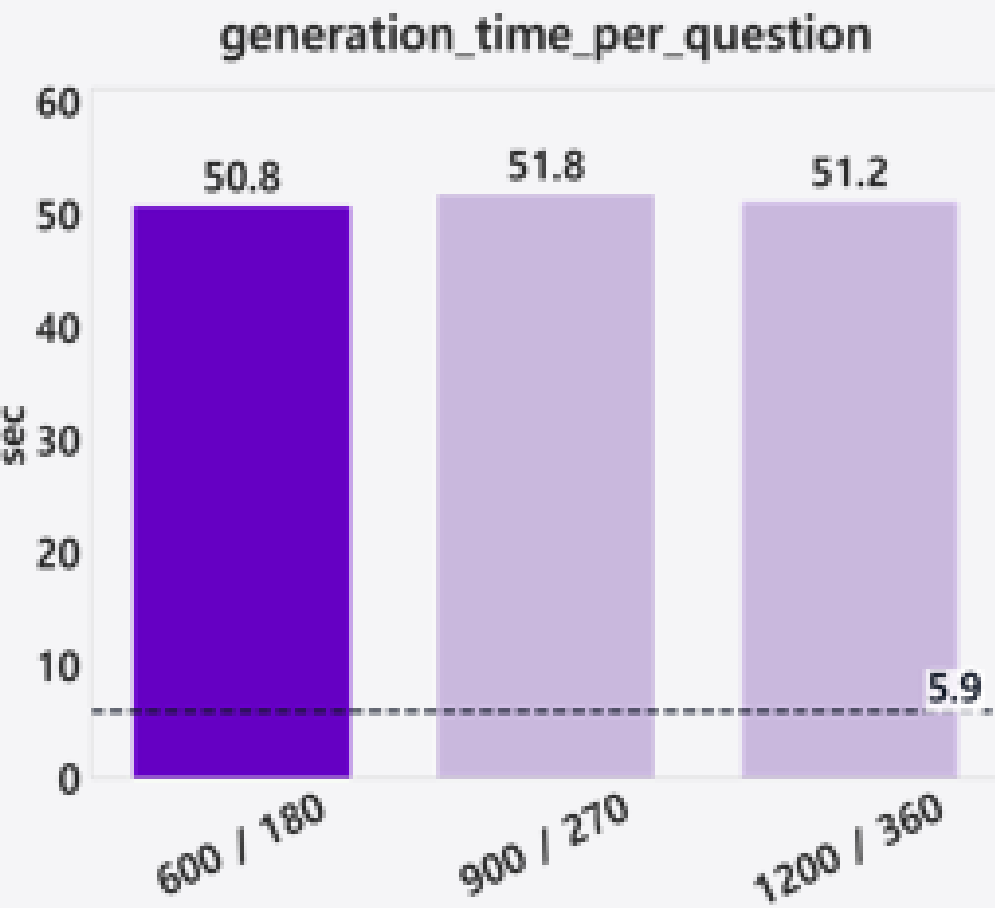
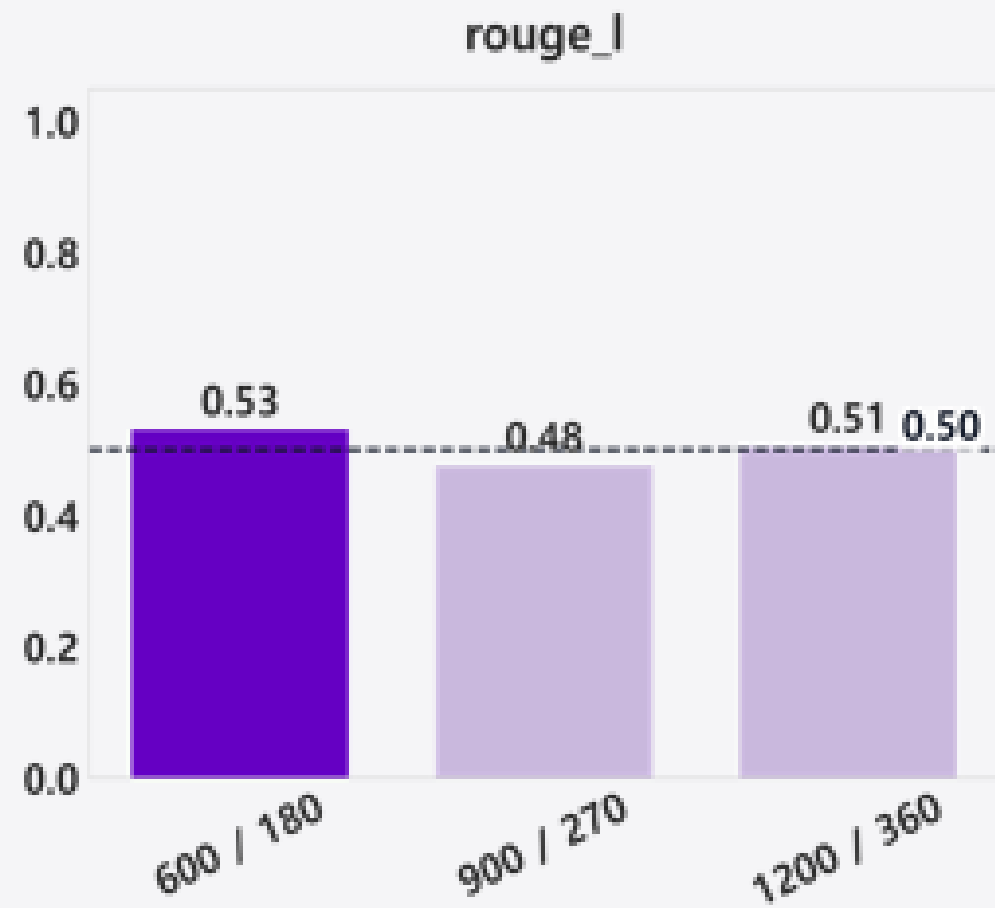
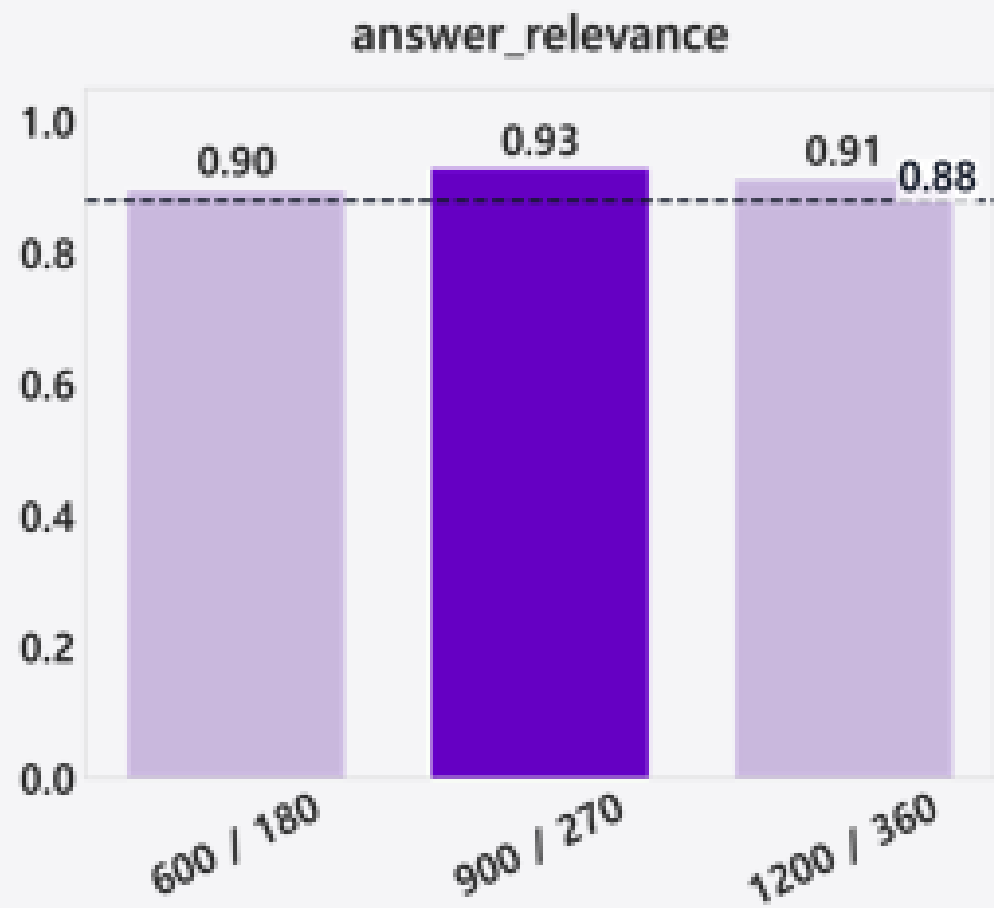
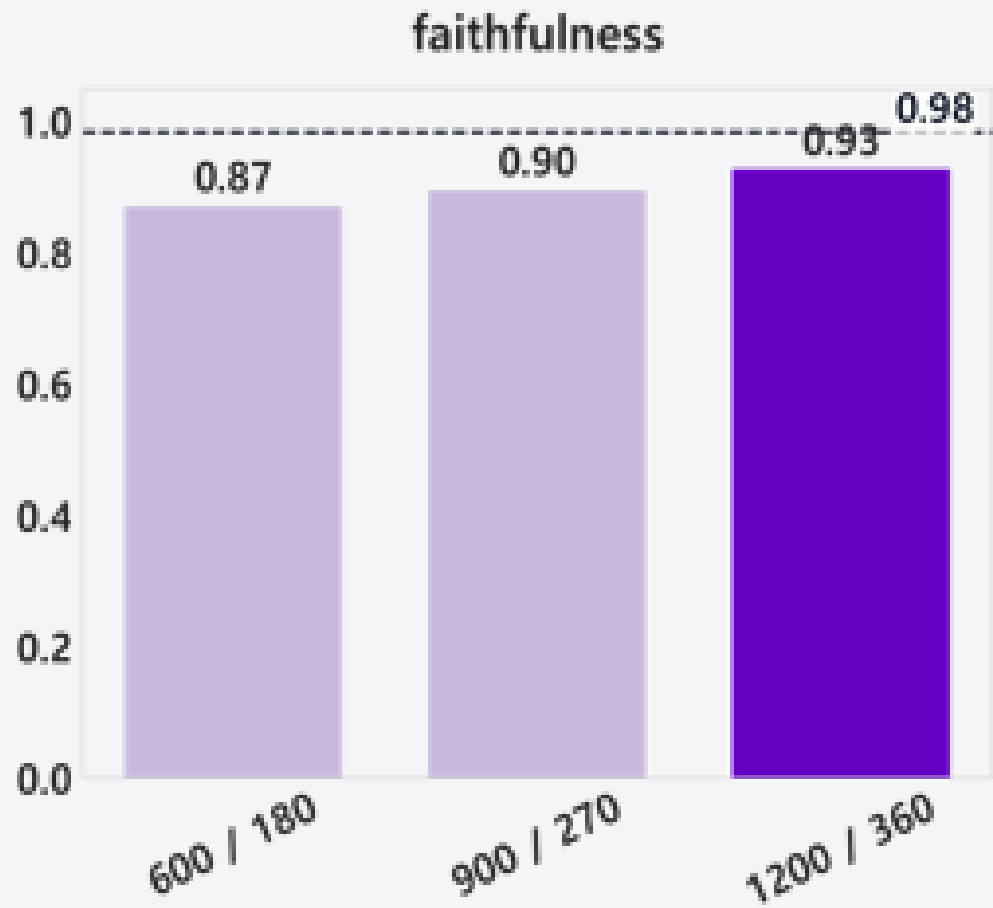
Size / Overlap (L4)	Retrieval				
	MRR	nDCG	Context Precision	Context Recall	Embedding Build Time
600 / 180	0.88	0.90	0.76	0.74	60.1
900 / 270	0.89	0.92	0.79	0.74	59.9
1200 / 360	0.90	0.90	0.80	0.76	56.8



4. 성능 개선 및 평가

6) Chunk size / Overlap

Size / Overlap (L4)	Generation			
	Faithfulness	Answer Relevance	Rouge I	Generation time per question
600 / 180	0.87	0.90	0.53	50.8
900 / 270	0.90	0.93	0.48	51.8
1200 / 360	0.93	0.91	0.51	51.2



Best strategy Objective

4. 성능 개선 및 평가

7) 최종 조합 실험

(A100)	Retrieval				
	MRR	nDCG	Context Precision	Context Recall	Embedding Build Time
600/180/256/10	0.86	0.91	0.76	0.73	31.7
900/270/256/10	0.86	0.92	0.79	0.74	31.2



4. 성능 개선 및 평가

7) 최종 조합 실험

(A100)	Generation			
	Faithfulness	Answer Relevance	Rouge I	Generation time per question
600/180/256/10	0.96	0.95	0.38	25.6
900/270/256/10	0.98	0.92	0.31	27.0



5. 비용 효율성 분석 및 확장 전략

OpenAI API vs A100 서버 비용 산정 기준

OpenAI API

1M tokens 당 단가

Text-Embedding-3-small	\$0.02
GPT-5-mini 입력	\$0.25
GPT-5-mini 출력	\$2.00

Embedding 142,922 / GPT 입력 1882 / GPT 출력 512 tokens

\$0.00429 / question

embedding 하나당 5개의 질문 가정

A100 서버

Google Console 기준 (VRAM 40GB)

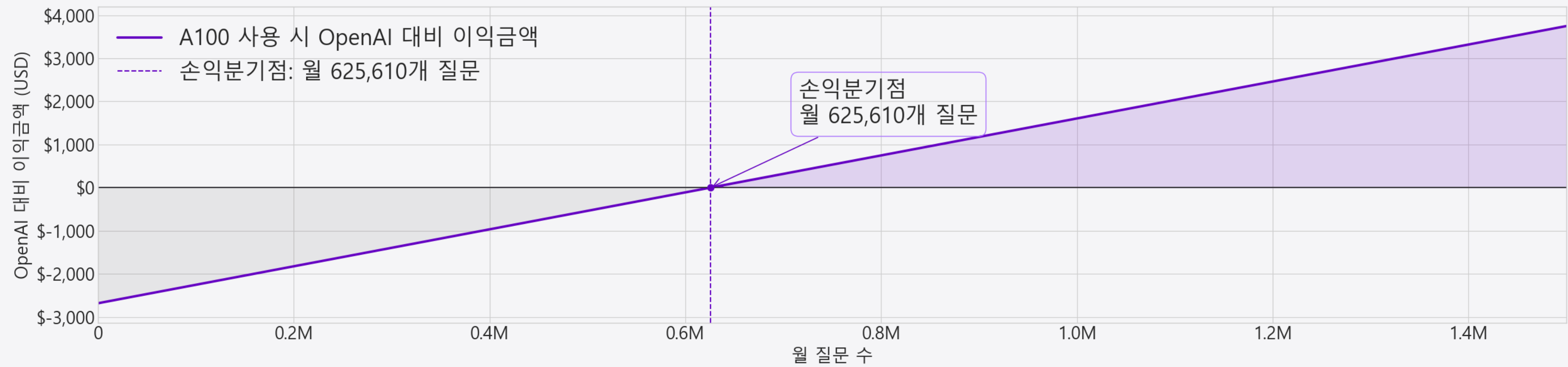
월 고정비

\$2,682.57 / month

OpenAI API는 질문 1건당 비용으로, A100 서버는 월 고정비로 계산

5. 비용 효율성 분석 및 확장 전략

손익분기점 계산



월 질문 수 (건)	OpenAI 비용 (/월)	A100 비용 (/월)	더 저렴한 방식	절약 금액 (/월)
100,000	\$429		OpenAI API	\$-2,254
625,610	\$2,683	\$2,683	동일 (손익분기점)	\$0
1,200,000	\$5,146		A100	\$2,463

월 질문 수가 약 62.6만 건을 넘으면 A100 서버 임대 방식이 OpenAI API보다 비용상 유리

5. 비용 효율성 분석 및 확장 전략

한계점 및 확장 전략

손익분기점 조건
월 625,610건
필요 처리 속도 : 4.14초 / question 이하

현재 한계점
처리 속도 : 32.4초 / question embedding 하나당 5개의 질문 가정
월 80,000건 처리 가능

확장 전략 : A100 비용 이점 확보를 위한 Batch 단위 처리

Batch Size	월 처리 가능량 (건)	손익분기점 대비	예상 절약 금액
1	80,000	13%	\$ -2,340
7.82	625,600	100%	\$0
16	1,280,000	205%	\$ 2,806

SUMMARY

목표 / 협업 프로세스

RAG 성능 목표 설정
실험·협업 체계 구축

데이터 분석 / 전처리

RFP 문서 특징 분석
PDF 변환·원문 추출

Baseline 모델 선정

세 가지 청킹 전략과
세 가지 모델 조합 비교

성능 개선 실험

목표치에 근접한
성능 도출 성공

비용 효율성 분석

모델 간 비용 비교
손익 분기점 계산

결론

오픈소스 모델 기반 목표 성능 달성

비용 효율화 가능성 확인

보완점

OCR·VLM 기반 문서 인식 고도화

Batch 처리 기반 서빙 구조 확장

경청해 주셔서 감사합니다!

감사합니다

1팀